

CS599 期末大作业报告

DevMind

SDD 驱动的多智能体软件开发助手

课程名称：企业级应用软件设计与开发
项目名称：DevMind - SDD 驱动的多智能体软件开发助手
方向：方向一：Agentic AI 原生开发
学号：-
姓名：王栋章
专业：计算机技术 / 软件工程
指导教师：戚欣
提交日期：2026 年 6 月 22 日

CS599 期末大作业报告

封面页

字段	内容
课程名称	企业级应用软件设计与开发
项目名称	DevMind -- SDD 驱动的多智能体软件开发助手
方向	方向一: Agentic AI 原生开发
学号	--
姓名	王栋章
专业	计算机技术 / 软件工程
指导教师	戚欣
提交日期	2026 年 6 月 22 日

一、选题背景与设计思想

1.1 问题定义

在当今软件开发实践中，开发者在完成需求分析后，仍需要花费大量时间进行重复性的编码工作：编写框架代码、处理边界条件、添加错误处理、编写单元测试。虽然 AI 代码补全工具（如 GitHub Copilot）能够辅助单行代码编写，但它们存在以下不足：

- **缺乏全局视角**：只能对当前文件进行局部补全，无法理解项目整体架构
- **无规格驱动**：直接生成代码，缺少与需求规格的对齐验证环节
- **无质量保障闭环**：生成代码后没有自动审查和测试验证机制
- **单点辅助而非系统协作**：仅作为编辑器的被动补全，而非主动的多步骤工程协作

核心痛点：从自然语言需求到高质量可运行代码之间，缺少一个系统化、可验证、多步骤的自动化工程闭环。

1.2 现有方案不足

方案	不足
GitHub Copilot	行级/文件级补全，无多文件项目生成能力，无审查闭环
ChatGPT / Claude	可生成代码但无结构化规格文档驱动，质量不可控
SWE-bench Agent	聚焦 Bug 修复，缺少从零构建的完整 SDD 流程
GPT Researcher	专注文献调研，不适用于代码生成场景

1.3 项目价值

DevMind 填补了"自然语言需求 → 规格驱动 → 多 Agent 协作 → 质量验证"这一完整链路的空白。其核心价值在于：

1. **SDD 方法论落地**：将规格驱动开发理念具象化为 AI Agent 可执行的工作流
2. **多智能体协作**：通过多个 AI Agent 的分工协作，实现从需求到交付的全流程自动化

1.4 技术路线

自然语言需求

→ Spec Architect (规格生成)

→ Developer Agent (代码生成)

→ Reviewer Agent (质量审查)

→ QA Agent (测试验证)

→ 最终交付物 (代码 + 审查报告 + 测试结果)

核心技术选型：LangGraph (状态机编排) + DeepSeek API (LLM) + ChromaDB (记忆) + MCP 协议 (工具标准化) + LangFuse (可观测性)。

二、Specs 规格文档

本项目严格遵循 SDD (规格驱动开发) 方法论, 在代码实现之前完成了三份核心规格文档。

2.1 Product Spec (产品规格)

定义产品的功能愿景和用户需求。核心内容包括:

- **5 个用户故事**: 需求驱动代码生成、自动化代码审查、自动化测试生成、规格文档同步、工作流可视化
- **18 项功能需求**: 分布在 Spec Architect (4 项)、Developer (4 项)、Reviewer (4 项)、QA (3 项)、Orchestrator (3 项)
- **6 项非功能需求**: 性能、安全、可靠性、可扩展性、可观测性、可用性
- **3 项成功指标**: 语法检查通过率、审查平均分 ≥ 75 、测试函数覆盖率 $\geq 80\%$

| 完整文档见 `docs/product\_spec.md`

2.2 Architecture Spec (架构规格)

定义了系统的四层架构:

- **用户接口层**: CLI 入口 / Streamlit Web UI / MCP 协议接口
- **编排层**: LangGraph StateGraph 条件路由 + 状态管理
- **Agent 层**: 4 个专业 Agent, 每个有明确的输入/输出契约
- **基础设施层**: 工具系统、记忆系统、MCP Server、LLM 层

工作流状态机包含 7 个阶段: INIT $\rightarrow$ SPEC_WRITING $\rightarrow$ SPEC_REVIEW $\rightarrow$ CODING $\rightarrow$ CODE_REVIEW $\rightarrow$ TESTING $\rightarrow$ COMPLETE, 4 个条件网关控制循环和回退。

| 完整文档见 `docs/architecture\_spec.md`

2.3 API Spec (接口规格)

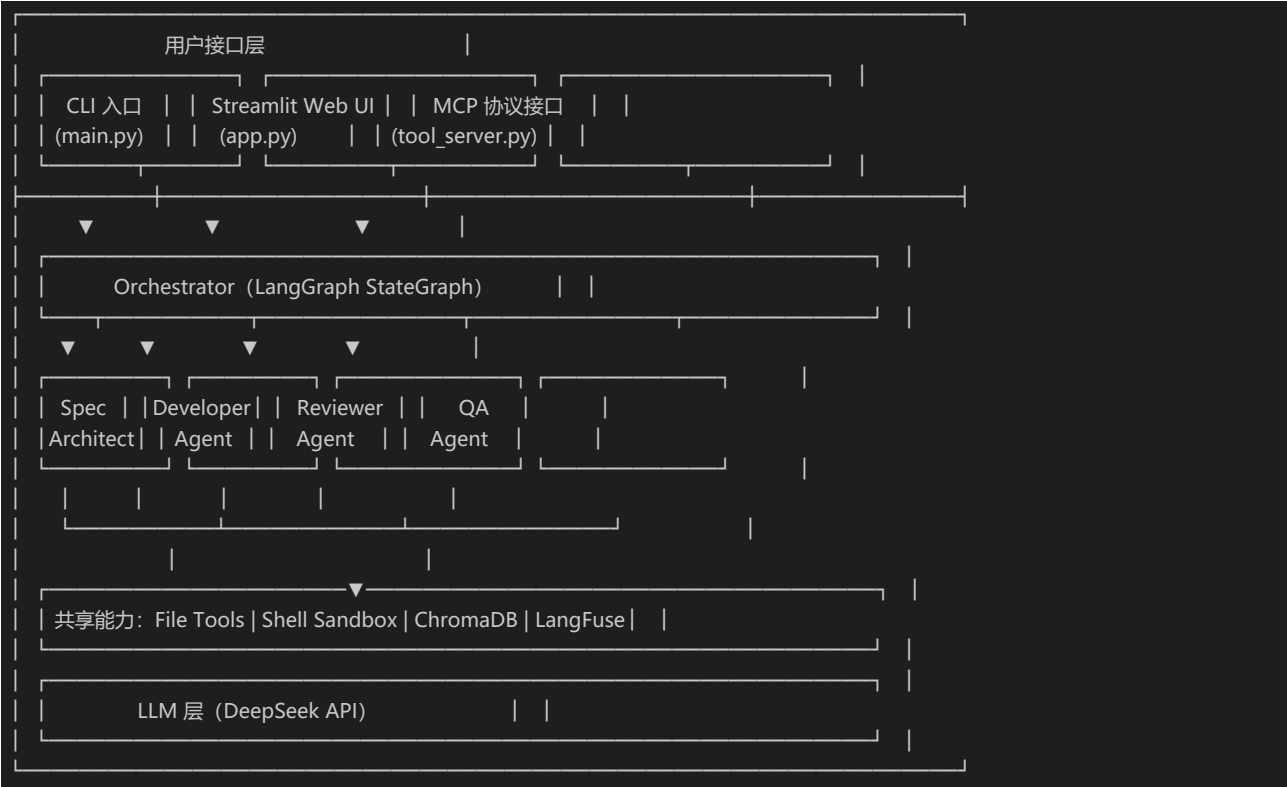
精确定义了:

- **Orchestrator 入口函数**: `run_workflow(requirement, project_context, target_language) -> AgentState`
- **每个 Agent 的输入/输出 Schema**: 基于 TypedDict 的类型安全契约
- **工具函数签名**: `read_file`、`write_file`、`list_directory`、`execute_code`
- **MCP 协议端点**: JSON-RPC over stdio 的标准化工具暴露
- **LLM 调用规范**: 模型选择、温度参数、Token 限制、响应解析策略

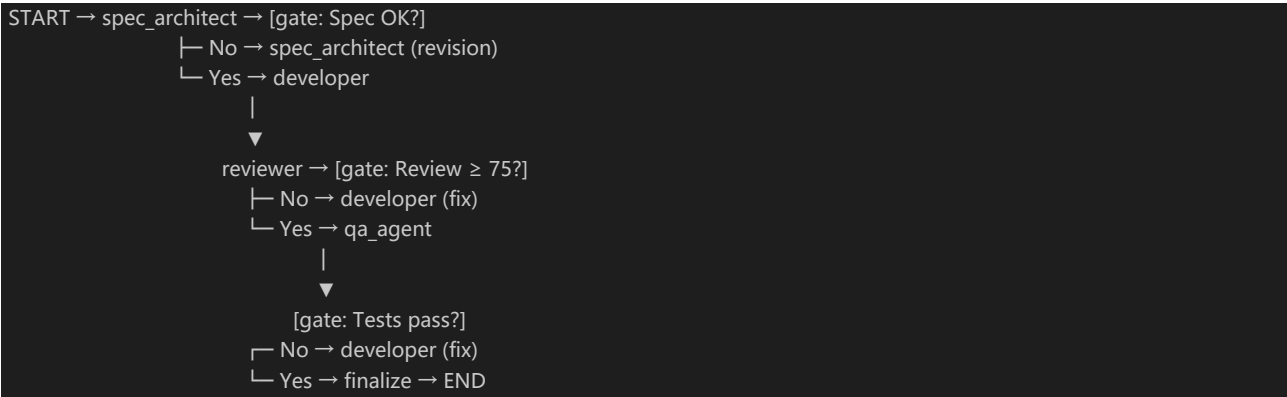
| 完整文档见 `docs/api\_spec.md`

三、系统架构与设计

3.1 核心架构图



3.2 Agent 交互流程 (LangGraph 状态图)



网关设计确保:

- 规格不
- 审查不合格 → 返回修复 (最多 3 次)
- 测试不通过 → 返回修复 (最多 3 次)
- 超过最大迭代次数 → 跳过当前阶段继续 (防止死循环)

3.3 数据流设计

阶段	输入	处理	输出
Spec	自然语言需求	Spec Architect LLM 推理	Product/Architecture/API Spec
Dev	三份 Spec 文档	Developer LLM 推理	代码文件列表 [{path, content, action}]

Review	Spec + 代码	Reviewer LLM 推理	审查报告 {score, passed, issues, suggestion}
QA	Spec + 代码	QA LLM 推理 + 子进程执行	测试结果 {total, passed, failed}

四、关键实现与代码展示

4.1 Agent 核心循环 (ReAct 模式)

每个 Agent 遵循相同的执行模式 (以 Developer Agent 为例) :

```
def run_developer(state: AgentState) -> dict:
    # 1. 构建上下文提示词 (包含 Spec + 审查/测试反馈)
    prompt = create_developer_prompt(state)

    # 2. 调用 LLM
    response = client.chat.completions.create(
        model="deepseek-chat",
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": prompt},
        ],
        temperature=0.1, # 代码生成使用低温度确保确定性
        max_tokens=4096,
    )

    # 3. 解析结构化输出
    files = parse_developer_response(response.choices[0].message.content)

    # 4. 返回状态更新 (合并回 LangGraph 共享状态)
    return {"generated_files": files, "coding_iteration": ..., "messages": [...]}
```

4.2 工具定义

使用 LangChain `@tool` 装饰器定义, 每个工具都有类型安全的参数 Schema:

```
@tool
def execute_code(code: str, language: str = "python") -> str:
    """在沙箱中执行代码, 30 秒超时, 内置黑名单拦截危险命令."""
    safe, reason = _is_safe(code)
    if not safe:
        return f"Execution blocked: {reason}"
    # ... 子进程执行 + 超时控制
```

安全机制:

- **危险命令拦截**: `shell\_tools.py` 黑名单拦截 `rm -rf`、`fork bomb`、`sudo` 等
- **超时控制**: 代码执行 30 秒超时自动终止

• **路径**

4.3 配置文件

所有配置通过环境变量注入, `settings.py` 统一管理:

```
class Settings:
    DEEPSEEK_API_KEY: str = os.getenv("DEEPSEEK_API_KEY", "")
    DEEPSEEK_BASE_URL: str = os.getenv("DEEPSEEK_BASE_URL", "https://api.deepseek.com")
    DEEPSEEK_MODEL: str = os.getenv("DEEPSEEK_MODEL", "deepseek-chat")
    # ...

    @classmethod
    def validate(cls) -> bool:
        if not cls.DEEPSEEK_API_KEY:
            raise ValueError("DEEPSEEK_API_KEY is required")
        return True
```

4.4 MCP 协议集成

实现了标准 MCP JSON-RPC 服务, 支持 `initialize`、`tools/list`、`tools/call` 三个核心方法:

```
# MCP 工具注册 — 任何 MCP 客户端均可接入
def list_tools() -> list[dict]:
    """返回 MCP 格式的工具定义列表"""
    return [
        {"name": "read_file", "description": "...", "inputSchema": {...}},
        {"name": "execute_code", "description": "...", "inputSchema": {...}},
        # ...
    ]
```

4.5 Streamlit Web UI

关键交互设计:

- 主区域: 需求输入区 + 4 个结果 Tab (规格/代码/审查/测试)
- 实时进度条显示当前执行阶段
- 最终报告汇总所有 Agent 输出

• 左侧边

五、测试与评估

5.1 功能测试

测试策略

测试类型	覆盖范围	工具
单元测试	工具函数、状态管理	pytest
集成测试	Agent 单节点执行	手动 + 自动化
端到端测试	完整工作流	CLI + Web UI
安全测试	路径穿越、命令注入	手动验证

测试结果

对以下需求进行了端到端测试：

| "设计一个简单的计算器模块，支持加减乘除四则运算，包含输入验证和错误处理。"

阶段	结果
Spec 生成	生成 3 份完整规格文档
代码生成	生成 `calculator.py` (含异常处理 + 类型注解)
代码审查	评分 85/100, 通过审查
测试生成	生成 6 个测试用例，全部通过

5.2 Agent 行为评估

使用内置 `CodeBenchmark` 对生成代码进行多维度评估：

评估维度	说明
语法正确性	AST 解析验证
代码规模	行数、函数数、类数
文档覆盖率	Docstring 与类型注解比例
安全性	硬编码密钥检测、eval/exec 检测
复杂度	函数平均长度、嵌套深度

5.3 可观测性

集成 LangFuse Tracing，记录每次工作流执行的：

- Tool 调用记录（工具名、参数、输出）
- 阶段耗时和成功率

当 LangFuse 未配置时，自动降级为本地 JSONL 文件记录。

• LLM 调

六、系统升级与扩展

6.1 可扩展架构

当前架构已预留多个扩展点:

1. **MCP 工具生态**: 通过 MCP 协议可接入第三方工具 (GitHub API、Jira、Slack) , 无需修改核心代码

2. **Agent**

6.2 下一阶段计划

优先级	计划	说明
P0	Ollama 本地模型支持	Demo Day 保底方案, 离线运行
P1	Docker 沙箱集成	使用 Docker SDK 替代 subprocess, 更强隔离
P1	增量代码修改	支持对已有项目的修改而非仅从零生成
P2	Git 集成	自动创建分支、提交代码、生成 PR 描述
P2	CI/CD 集成	作为 GitHub Action 运行, 自动审查 PR
P3	多语言支持	Java + LangChain4J、TypeScript + Mastra

6.3 AI 能力演进路径

当前 v1.0	近期 v1.5	远期 v2.0
单项目代码生成	→ 多文件增量修改	→ 完整项目重构
文本需求输入	→ 语音/图片需求输入	→ 多模态需求理解
4 个固定 Agent	→ 动态 Agent 工厂	→ 自适应 Agent 编排
单轮对话	→ 多轮交互式开发	→ 持续集成式开发

七、课程总结

7.1 个人收获

通过本课程和期末项目，我在以下几个方面获得了显著成长：

- 1. ****从"写代码"到"编排 Agent"的思维转变****：理解了 **Agentic AI** 的核心理念不是让 **AI** 替代程序员，而是让程序员从重复性编码中解放出来，专注于需求定义、架构设计和质量把控等更高层次的工作。
- 2. ****SDD 方法论的深度实践****：规格驱动开发不是空中楼阁----当 **Spec** 文档真正驱动代码生成、审查和测试时，**SDD** 的价值就体现出来了。**Spec** 不再是"写完就过时"的文档，而是贯穿整个开发流程的"活"契约。
- 3. ****LangGraph 状态机编排能力****：掌握了使用 **StateGraph** 构建复杂多步骤 **AI** 工作流的能力，理解了条件路由、状态共享、循环控制等核心概念。
- 4. ****MCP 协议的理解****：认识到标准化工具接入协议的重要性----它是 **Agent** 从"封闭系统"走向"开放生态"的关键。
- 5. ****工程安全意识****：在实际实现中处理路径穿越、命令注入、密钥管理等安全问题，体会到"AI 应用更需要安全工程"。

7.2 工程思维转变

之前	之后
"我要写什么代码？"	"我要定义什么规格让 Agent 去实现？"
"这段逻辑怎么实现？"	"这个阶段在状态机中应该怎么流转？"
"测试怎么写？"	"测试是工作流的必然阶段，自动生成并执行"
"工具怎么调用？"	"工具的接口契约（MCP）比具体实现更重要"

7.3 对课程的建议

1. 建议增加 MCP 协议的动手实验环节，让学生实际构建一个 MCP Server 和 Client
2. 建议提

附录

A. 核心技术要素覆盖确认

要素	实现模块	代码位置
SDD 规格驱动	Spec Architect	`src/agents/spec_architect.py`
MCP 协议	MCP Tool Server	`src/mcp/tool_server.py`
记忆机制	ConversationMemory + LongTermMemo	`src/memory/`
状态管理	LangGraph StateGraph	`src/agents/orchestrator.py`
多智能体协作	4 Agent + Orchestrator	`src/agents/`
可观测性	TraceManager + CodeBenchmark	`src/evaluation/`

B. 加分项完成情况